Face Mask Detector

Real-time face mask detection using Deep Learning and OpenCV

Digital Image Processing and Computer Vision
Submitted to: Dr. Mahmoud Alshewimy
Team Members: Ahmed Mohammad Fayad | Ahmed Alaa Eldin Abdelrahman Kamel

1. Project Overview

The Face Mask Detector is a computer vision application that identifies whether a person is wearing a face mask in real time. It operates through a two-stage pipeline: first, a Haar Cascade classifier detects face regions within a video frame; second, a Convolutional Neural Network (CNN) classifies each detected face as masked or unmasked. The project was developed iteratively across four phases — from a single-file baseline to a fully modular, deployed application — and includes a live web demo accessible through HuggingFace Spaces.

2. Algorithms and Approach

2.1 Face Detection — Haar Cascade

Face localization uses OpenCV's Haar Cascade classifier, a classical (non-deep-learning) method trained on Haar-like features — patterns of light and dark rectangles characteristic of frontal human faces. It scans the image at multiple scales using a sliding window, and a region is confirmed as a face only when multiple neighboring windows agree. This approach is fast and requires no GPU, making it suitable for real-time operation on a standard laptop.

2.2 Mask Classification — CNN

Each detected face region is preprocessed and fed into a binary classifier. Three model architectures were implemented and compared:

- Custom CNN — built from scratch with 3 convolutional blocks, enhanced with BatchNormalization and Dropout for regularization.
- MobileNetV2 — transfer learning using a model pre-trained on ImageNet. The base is frozen and a small classifier head is trained on the mask dataset.
- VGG16 — a second transfer learning baseline using the heavier VGG16 architecture, included for comparison against MobileNetV2.

2.3 Preprocessing Pipeline

The following preprocessing steps are applied to each face crop before classification:

- Resize to 150x150 pixels (model input size).
- BGR to RGB conversion (OpenCV reads BGR; the model expects RGB).
- Histogram equalization on the luminance channel for lighting robustness.
- Pixel normalization from [0, 255] to [0.0, 1.0].
- Batch dimension expansion for model compatibility.

2.4 Training Configuration

All models were trained using the same dataset and augmentation settings:

- Optimizer: Adam with binary cross-entropy loss.
- Epochs: 10 | Batch size: 16.
- Augmentation: rescaling, shear, zoom, and horizontal flip (training set only).
- Transfer learning models used a lower learning rate (1e-4) to avoid overwriting pre-trained weights.

3. Dataset

The dataset contains labeled face images divided into two classes: with_mask and without_mask. Images are split into training and test sets and stored in a directory structure that Keras reads automatically via ImageDataGenerator.

Split	with_mask	without_mask
Training	657 images	657 images
Test	97 images	97 images
Total	**754 images**	**754 images**

4. Results

The three models were evaluated on the same 194-image test set after 10 training epochs:

Model	Training Accuracy	Validation Accuracy	Type
MobileNetV2	**99.16%**	**98.97%**	Transfer
VGG16	95.21%	97.94%	Transfer
Custom CNN	96.58%	94.85%	Scratch

MobileNetV2 achieved the highest accuracy on both training and validation sets while remaining the most lightweight model — making it the best choice for real-time deployment. Transfer learning consistently outperformed the custom CNN trained from scratch, which is expected given the relatively small dataset size (~1,500 images). Notably, VGG16 achieved higher validation accuracy (97.94%) than training accuracy (95.21%), suggesting strong generalization despite its size.

5. Technology Stack

Technology	Role
TensorFlow / Keras	Deep learning framework for building and training CNN models
OpenCV	Computer vision library for webcam capture and Haar Cascade face detection
MobileNetV2	Lightweight pre-trained model used for transfer learning
VGG16	Classic deep CNN used as a second transfer learning baseline
Streamlit	Python web framework used to build the interactive dashboard
HuggingFace Spaces	Cloud platform used for deploying the Streamlit application
NumPy / Pillow	Image array manipulation and format conversion
Git / GitHub	Version control and project documentation across 4 development phases

6. Deployment

The project is deployed as an interactive Streamlit web application on HuggingFace Spaces, accessible to anyone without any local setup. Users can upload an image and the app detects all faces, classifies each as masked or unmasked, displays annotated bounding boxes with confidence percentages, and shows a compliance summary. The model selector in the sidebar allows switching between all three trained models live.

Live Demo: https://huggingface.co/spaces/Mr0Diablo/face-mask-detector